Obchodní akademie, Vyšší odborná škola a Jazyková škola
s právem státní jazykové zkoušky Uherské Hradiště

MATURITNÍ PRÁCE

2D AKČNÍ HRA

2026

Stanislav Vacula 4. B

Vnitřní předpis OA, VOŠ a JŠ, čj. 025/ORG/2023

Příloha č. 3

ZADÁNÍ MATURITNÍ PRÁCE INFORMAČNÍ TECHNOLOGIE

Jméno žáka:	Stanislav Vacula
Téma maturitní práce:	2D akční hra
Vedoucí práce:	Mgr. Karel Jindra
Oponent práce:	Ing. Martin Šimůnek
Způsob zpracování:	Žák vytvoří 2D akční hru – hra bude rozdělená na úrovně, ve kterých se bude bojovat s nepřáteli a bude obsahovat logické prvky, které umožní projít některé úrovně jednoduše. Hra bude vyvinuta v enginu GODOT v jazyce GDScript. Základem hry bude hlavní postava, která se bude pohybovat a bojovat s monstry. Hra bude ve stylu hry Cactus McCoy. Hra nabídne několik odlišných levelů, které budou mít vždy více různých řešení a nikdy se nebudou opakovat. Grafika bude v 2D stylu (nejspíše pixel art). Hra bude obsahovat zvukové efekty (například zvuk boje atd.) Hra bude otestována minimálně dvěma testery a poznatky z testování budou uvedeny v dokumentaci. Výstupem bude plně hratelná hra pro Windows.
Pokyny k průběhu vypracování:	Práci bude žák vést v repositáři na GitHubu či obdobném včetně historie práce od zadání až po odevzdání projektu. Zdroje informací žák uvede dle normy ČSN ISO 690. Pokud je zdrojem informací AI, žák je cituje dle vnitřní směrnice čj. 025/ORG/2023 školy.

Pokyny k odevzdání:	**Žák odevzdá práci v tištěné a elektronické podobě** • Tištěná podoba práce obsahuje uživatelskou a technickou dokumentaci. **Tištěnou podobu** (v kroužkové vazbě) žák odevzdá na studijní oddělení školy. **Elektronická podoba práce obsahuje** • Dokumentaci ve formátu PDF/A • Resumé ve formátu PDF/A • Výsledný projekt, zdrojové soubory a potřebné knihovny pro spuštění projektu • Prezentaci projektu **Elektronická podoba** práce se nahrává do IS školy dle pokynů vedoucího práce nebo vedení školy. V případě, že se jedná o projekt, na kterém pracovalo více žáků, je povinnou součástí dokumentace podrobné rozdělení činností při práci na projektu.
Kritéria hodnocení:	Hodnocení se skládá z celkové kvality zpracování práce, dokumentace, z kvality prezentace při obhajobě práce, diskuse a z průběžného hodnocení žáka v rámci kontrolních dnů.
Obhajoba projektu	Obhajoba projektu se skládá ze dvou části – prezentace projektu (včetně podpůrné elektronické prezentace) a diskuse nad řešením. Celková délka obhajoby je 20 minut, délka prezentace projektu by neměla překročit 10 minut.

30 -09- 2025

.....

Datum

.....

Podpis ředitele školy

Obchodní akademie, Vyšší odborná škola
a Jazyková škola s právem státní
jazykové zkoušky Uherské Hradiště
Nádražní 22
686 01 Uherské Hradiště

Prohlášení:

Souhlasím s tím, že s výsledky mé práce může být naloženo podle uvážení vedoucího maturitní práce a ředitele školy. V případě publikace budu uveden jako spoluautor. Prohlašuji, že jsem na celé maturitní práci pracoval samostatně a veškeré použité zdroje jsem citoval.

V Uherském Hradišti, dne *30. 3. 2026*

.....

podpis absolventa

RESUMÉ

Moje maturitní práce je 2D akční hra s názvem Lone Survivor, která je vytvořena v herním enginu Godot 4.4. Hra sleduje příběh vojáka, jenž po pádu vrtulníku musí přežít v nepřátelském území plném pastí a ozbrojených protivníků. Cílem projektu je ukázat propojení herního designu, programování a animace v jednotném funkčním celku a vytvořit hratelný prototyp, který demonstruje tvorbu herní logiky, pohybu, boje a interakce s prostředím v moderním vývojovém nástroji.

OBSAH

ÚVOD **8**

1 TEORETICKÝ ÚVOD **9**

1.1 Představení projektu 9

1.2 Použité technologie 9

1.2.1 Herní Engine – GODOT 9

1.2.2 Grafika hry - Pixelorama/Photoshop 9

1.2.3 Programovací jazyk a nástroje 10

1.3 Konkurenční řešení 10

1.3.1 Analýza konkurenčních řešení 10

1.3.2 Analyzované konkurenční řešení 10

1.3.3 Shrnutí analýzy 11

2 KONCEPT A DESIGN HRY **12**

2.1 Herní design 12

2.1.1 Charakteristika pohybu 12

2.1.2 Soubojový systém a AI nepřátel 12

3 TECHNICKÁ REALIZACE **13**

3.1 Struktura projektu a základní třídy 13

3.1.1 Architektura scén a využití vestavěných tříd 13

3.1.2 Herní smyčka a zpracování fyziky 14

3.1.3 Level design a tvorba prostředí 14

3.2 Tvorba grafických assetů 16

3.3 Programování pohybu a animací 17

3.4 Detekce kolizí a interakce 18

3.5 Implementace UI a stavů hry 19

3.5.1 Heads-Up Display (HUD): 20

3.5.2 Systém stavů hry: 20

3.6 Implementace audia 22

3.7 Systém herních scén a hierarchie uzlů 23

3.8 Implementace systému deníku 24

3.9 Implementace dynamických bariér a detekce hráče 26

3.9.1 Technické řešení Backtrack Triggeru 26

3.10 Funkce export 26

3.10.1 Nastavení rychlosti střelby 26

3.11 Konfigurační nastavení systému 27

4 UŽIVATELSKÁ ČÁST 28
4.1 Návod ke spuštění hry 28
4.2 Uživatelské rozhrání po spuštění hry 28
4.2.1 Main menu hry 29
4.2.2 Diary menu 30
4.2.3 Controls menu 31
4.2.4 Pause menu 32
4.2.5 Win/Die menu 33
4.3 Průchod hrou 34
4.4 Minimální systémové požadavky 38
5 TESTOVÁNÍ 39
5.1 Metodika testování 39
5.1.1 Tester 1 (Zkušený hráč, spolužák) 39
5.1.2 Tester 2 (Začátečník, příbuzný) 39
5.1.3 Tester 3 (Vývojář her) 39
5.2 Odstranění chyb (bugů) 39
5.3 Výsledek testování 40
6 REPOZITÁŘ S ŘEŠENÍM PRÁCE 41
ZÁVĚR 42
SEZNAM POUŽITÉ LITERATURY 43
SEZNAM OBRÁZKŮ 45
SEZNAM TABULEK 46

ÚVOD

Herní průmysl se v posledních letech rychle rozvíjí a díky moderním nástrojům dnes může vlastní hru vytvořit i jeden člověk. Jedním z těchto nástrojů je herní engine Godot 4.4, který umožňuje poměrně jednoduše vytvářet 2D i 3D hry. Tato maturitní práce se zaměřuje na návrh a vytvoření 2D akční videohry s názvem „Lone Survivor“.

Cílem práce bylo vytvořit funkční hru pro operační systém Windows, která kombinuje programování, herní design a vlastní grafiku. Hra je vytvořena v programovacím jazyce GDScript a běží v enginu Godot. Výsledkem projektu je samostatná aplikace, ve které se hráč snaží přežít v nepřátelském prostředí a postupně projít herní úrovně.

Grafická stránka hry je vytvořena ve stylu pixel art. Tento styl vychází ze starších videoher, ale i dnes je mezi hráči velmi populární. Pro tvorbu grafiky, například animací postavy nebo prostředí, byly použity programy Pixelorama a Photoshop. Samotná hra byla vytvořena v enginu Godot, který umožňuje pracovat s fyzikou, animacemi, logikou hry i uživatelským rozhraním.

Důležitou částí hry je pohyb postavy a interakce s prostředím. Herní mapy jsou vytvořeny pomocí systému TileMap, který umožňuje skládat úroveň z menších grafických dlaždic a zároveň nastavit jejich kolize. Hráč se pohybuje v terénu s různými překážkami, kde musí skákat, vyhýbat se nepřátelům a bránit se pomocí střelby. Inspirací pro pohyb a souboje byla hra Cactus McCoy.

Ve hře se nachází také nepřátelé s jednoduchou umělou inteligencí. Ta funguje na principu stavového automatu. Nepřátelé dokážou hráče detekovat pomocí uzlu RayCast2D, který funguje jako neviditelný paprsek představující jejich zorné pole. Pokud paprsek narazí na hráče a není mezi nimi překážka, nepřítel začne útočit. Aby hra nebyla příliš obtížná, obsahuje také systém dobíjení zbraně (cooldown), který dává hráči čas reagovat nebo se schovat.

Součástí hry je také jednoduché uživatelské rozhraní (GUI), které hráči zobrazuje například stav zdraví. Před dokončením projektu byla hra otestována několika nezávislými testery. Jejich zpětná vazba pomohla odhalit některé chyby a upravit obtížnost některých částí hry.

Práce na tomto projektu mi umožnila lépe pochopit, jak funguje vývoj her. Naučil jsem se pracovat s herním enginem, programovat herní logiku a vytvářet vlastní grafiku. Tyto zkušenosti mohou být užitečné pro další studium nebo práci v oblasti informačních technologií.

1 TEORETICKÝ ÚVOD

1.1 Představení projektu

Projekt se zaměřuje na vývoj vlastní 2D akční plošinové hry. Projekt spadá do oblasti vývoje počítačových her. Cílem je navrhnout a vytvořit funkční hru s propracovanými herními mechanikami, plynulým ovládáním a vyváženou obtížností pro všechny hráče.

1.2 Použité technologie

Při vývoji hry bylo potřeba zvolit technologie, které umožňovaly projekt realizovat. Při výběru byl hlavní důraz kladen na populární, aktualizované technologie a s kvalitní dokumentací. Dalšími důležitými faktory výběru byla přehlednost frameworku a bohatý systém hotových řešení.

1.2.1 Herní Engine – GODOT

- **Název:** Godot Engine
- **Verze:** 4.4
- **Autor:** Godot Community
- **Licence:** MIT licence (open-source)
- **Dostupnost:** https://godotengine.org/download

Hra je tvořena v enginu GODOT, který má vlastní vestavěný skriptovací jazyk GDScript, který je syntaxí podobný jazyku Python. Při výběru pro mě bylo klíčové, aby mě engine nebrzdil, ale naopak mi pomáhal. Godot jsem zvolil hlavně proto, že je to moderní engine s výbornou dokumentací, kde člověk nemusí každou drobnost hodinu vyhledávat. Navíc je to open-source, což dodává jistotu, že v budoucnu nepřijdou žádné skryté poplatky nebo změny licencí.

1.2.2 Grafika hry - Pixelorama/Photoshop

Pixelorama

- **Název:** Pixelorama
- **Autor:** Orama Interactive
- **Licence:** Open-Source
- **Dostupnost:** https://orama-interactive.itch.io/pixelorama

Pixelorama je nástroj pro tvorbu pixel art grafiky a animací. Byl využit především pro tvorbu animací postavy.

Adobe Photoshop

- **Název:** Adobe Photoshop
- **Autor:** Adobe Inc.
- **Licence:** Komerční software
- **Dostupnost:** https://www.adobe.com/products/photoshop.html

Adobe Photoshop byl využit pro tvorbu grafických prvků prostředí a úpravu obrázků.

1.2.3 Programovací jazyk a nástroje

- **Programovací jazyk:** GDScript
- **Vývojové prostředí:** Godot Editor
- **Verzovací systém:** GitHub

Pro správu verzí a ukládání projektu byl využit systém Git a platforma GitHub.

1.3 Konkurenční řešení

1.3.1 Analýza konkurenčních řešení

Pro vývoj vlastní hry jsem potřeboval analyzovat, jak fungují hry podobného principu, abych věděl, jaké mechaniky jsou běžné a co hráči mohou od mé hry očekávat. Cílem analýzy bylo zjistit jaké podobné produkty se již nachází na trhu, zjistit jakým způsobem řeší ovládaní, soubojový systém a návrh úrovní.
Cílem analýzy bylo zjistit silné a slabé stránky konkurenčních her a zároveň díky ní zjistit, v čem může být má hra odlišná.

1.3.2 Analyzované konkurenční řešení

Cactus McCoy

Jedná se o 2D akční plošinovou hru, ve které hráč ovládá prokletého kovboje proměněného v kaktus. Hra kombinuje boj zblízka, sbírání zbraní a plošinové prvky. Ve hře se nachází více úrovní.
Silné stránky této hry jsou plynulé animace, dynamický soubojový systém, dobře navržené levely. Slabé stránky hry jsou lehká obtížnost pro zkušenější hráče, použité starší technologie (hra byla tvořena pomocí Flash).

Shovel knight

Jedná se o 2D akční plošinovou hru, ve které hráč ovládá rytíře vyzbrojeného lopatou. Hra kombinuje boj zblízka, překonávání plošinových překážek a souboje s unikátními bossy. Hráč během hry sbírá herní měnu, za kterou si může odemykat vylepšení vybavení a schopností. Ve hře se nachází více samostatných úrovní.
Silné stránky této hry jsou přesné a responzivní ovládání, originální a propracované souboje s bossy.
Slabé stránky hry jsou vyšší obtížnost, která pro nezkušené hráče může být náročná, a jednodušší retro grafické zpracování, které nemusí vyhovovat všem moderním hráčům.

1.3.3 Shrnutí analýzy

Analyzované hry jsou postavené na podobném principu. Jedná se o 2D plošinovky s bojem proti nepřátelům a postupem úrovněmi. Důraz těchto her je kladen na plynulé ovládání a postupné zvyšování obtížnosti.

2 KONCEPT A DESIGN HRY

2.1 Herní design

V této kapitule je popsán návrh herních mechanismů a způsob, jakým je realizována hratelnost s využitím nástrojů enginu Godot.

2.1.1 Charakteristika pohybu

Hlavním cílem u pohybu postavy bylo dosáhnout responzivního pohybu postavy, který je pro 2D plošinovky důležitý. Cílem bylo dosáhnout toho, aby se postava ovládala příjemně a hráč měl pocit, že má nad postavou plnou kontrolu. Pro realizaci jsem využil přednastavené fyzikální metody, které engine nabízí. Klíčovou roli hraje metoda _physics_process(delta), která se stará o pravidelné vyhodnocování fyziky nezávisle na snímku hry.
V rámci této metody jsem využil vestavěnou metodu move_and_slide(). Tato metoda je součástí třídy CharacterBody2D a zajišťuje interakci s okolím (např. detekce podlahy). Po využití těchto dvou metod jsem si vytvořil proměnné pro rychlost chůze (SPEED) a délku výskoku, čímž jsem vytvořil základní dynamiku pohybu postavy. Pro úvodní nastavení parametrů po spuštění hry jsem využil metodu _ready(), která zajišťuje inicializaci postavy a jejího okolí.

2.1.2 Soubojový systém a AI nepřátel

Soubojový systém hry je postaven na interakci hráče s protivníky. Nepřátelé využívají přepínání stavů, mezi režimem hlídkování a útokem, které jsem naprogramoval. Pro detekování hráče jsem využil přednastavený uzel RayCast2D. Tento uzel je neviditelná přímka mezi nepřítelem a hráčem. Ve svém skriptu kontroluji, zda tato přímka narazila na hráče, nebo mu stojí v cestě nějaká překážka (např. zeď). Díky této metodě nepřítel nevidí postavu skrze pevné objekty. U dynamiky boje jsem se snažil, aby hra nebyla nefér, proto jsem ve skriptu vytvořil časovač výstřelů (SHOOT_COOLDOWN). Tím jsem zajistil, že nepřítel po detekci hráče střílí v pravidelných intervalech, což dodává hráči prostor pro schování se, nebo vyhnutí se výstřelu.

3 TECHNICKÁ REALIZACE

Tato kapitola je zaměřena na popsaní technického pozadí aplikace, strukturu skriptů a využití přednastavených tříd frameworku Godot, které výrazně usnadňují vývoj herních mechanik.

3.1 Struktura projektu a základní třídy

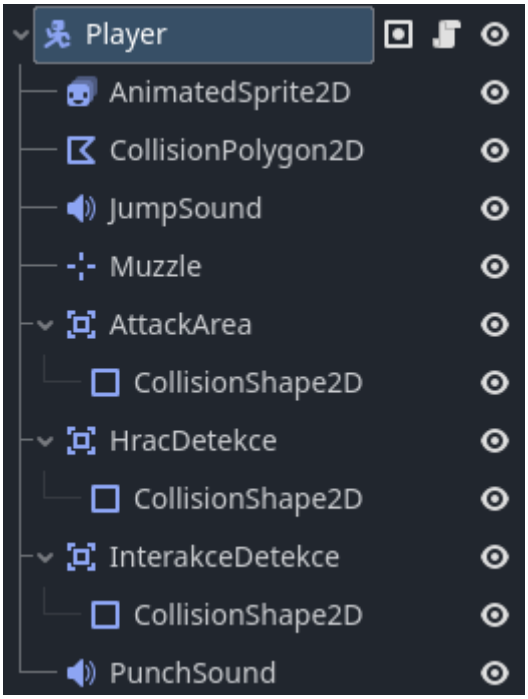
Tato kapitola popisuje, jak jsou v mé hře využity základní stavební prvky enginu Godot – tzv. uzly (Nodes) a scény – a jak jejich vzájemné propojení vytváří fungující herní svět. Zaměřuji se zde především na využití vestavěných funkcí, které Godot nabízí pro usnadnění práce s fyzikou a grafikou.

3.1.1 Architektura scén a využití vestavěných tříd

Godot je objektově orientovaný framework, kde základem jsou třídy. Tyto třídy již v sobě mají naprogramovanou složitou logiku, kterou lze využít bez nutnosti psaní kódu pro fyziku nebo vykreslování.

- **CharacterBody2D:** Pro hlavní postavu i nepřátele jsem zvolil třídu CharacterBody2D. Jedná se o předpřipravený herní objekt, který má v enginu již před programované vlastnosti pro pohyb ve 2D prostoru. Klíčová je metoda move_and_slide() – Tato metoda řeší detekci kolizí, klouzání podél stěn a pohyb po nakloněných rovinách
- **RayCast2D:** Tento uzel funguje jako neviditelná přímka, která je vysílaná z bodu A do bodu B. Tato třída se využívá pro účely detekce objektů. Metodu jsem použil pro detekci viditelnosti hráče. Skript přebere informaci, že nepřítel narazil na hráče, další informace jsou popsány ve skriptu.

- **AnimatedSprite2D:** Tato třída slouží k správě 2D animací v Godotu. Godot v tomto uzlu nabízí rozhraní pro import obrázků a jejich řazení do sekvencí animace. Ve skriptu pouze přepínám mezi těmito stavy pomocí metody .play().



Obr. 1 Ukázka stromu scén u hlavní postavy

Zdroj: Vlastní

3.1.2 Herní smyčka a zpracování fyziky

Základem celé hry je metoda _physics_process(delta). Tato metoda je v frameworku Godot již definována a navržena tak, aby byla enginem v pevných časových intervalech volána.
Využití parametru delta zajišťuje plynulost: reprezentuje čas hry v sekundách mezi jednotlivými kroky. Díky tomu, je zaručeno, že fyzikální interakce (skoky, střely) budou stabilní bez ohledu na výkon počítače.

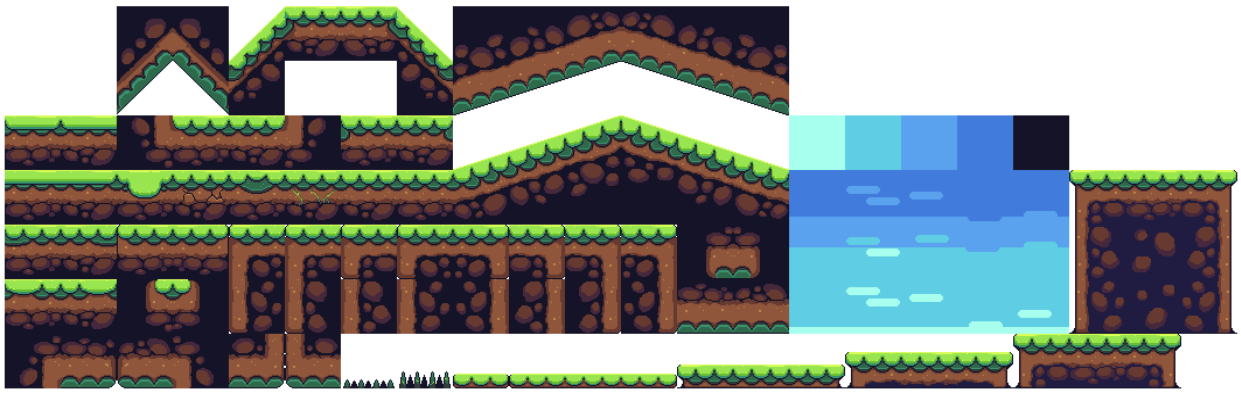
3.1.3 Level design a tvorba prostředí

Tato podkapitola se věnuje struktuře herních úrovní a technickému postupu při tvorbě v editoru Godot.
Při tvorbě úrovní jsem se zaměřil na metodu postupného zvyšování obtížností. První část mapy slouží jako tutoriál, kde se hráč naučí základní ovládání bez zásadního přímého ohrožení. Využil jsem systém **TileMap.** Pro zvýšení dynamiky prostředí jsem implementoval interaktivní objekty, jako jsou páky, které pomocí signálů (Signals) ovládají padací mosty. Tímto způsobem vzniká propojený herní svět, který vyžaduje od hráče nejen dobrý postřeh, ale i logické uvažování.

3.1.3.1 TileMap

Pro tvorbu herního světa je zvolen systém TileMapLayer. Tato metoda funguje tak, že rozdělí herní prostředí na pravidelnou mřížku, do které se již vkládají hotové jednotlivé dlaždice (bloky) z předpřipraveného souboru, tzv. Tilesetu. Tuto metodu jsem zvolil z těchto dvou hlavních důvodů:

- **Optimalizace:** Engine vykresluje mapu jako celek, což šetří výkon procesoru oproti vkládání samostatných obrazk
- **Efektivita:** V editoru Tilesetu jsem každé dlaždici (např. tráva, kámen, plošina) nadefinoval vlastní Collison Shape. Díky tomu jsem nemusel kolize programovat ručně pro každou část mapy



Obr. 2 Použitý TileSet

Zdroj: https://assetstore.unity.com/packages/2d/environments/nature-pixel-art-base-assets-free-151370.

3.1.3.2 Vizuální vrstvy (Parallax background)

Pro tvorbu pozadí je zvolen systém Parallax background, který je v godotu implementován. Systém spočívá v tom, že pozadí může mít více vrstev a zároveň se dá pozadí na sebe navázat, díky tomu pozadí pokračuje „do nekonečna“.

V Godotu existují dva uzly, které to umožňují, a to ParallaxBackground a ParallaxLayer. Vrstva má nastaven Motion Scale, který určuje poměr rychlosti pohybu vůči kameře. Díky tomu scéna nepůsobí staticky.

3.1.3.3 Struktura úrovní

Hra je logicky rozdělena do samostatných scén. Každá úroveň (Level) je v Godotu uložena jako samostatný soubor .tscn. Toto umožňuje snadnou správu a opětovné využití objektů. Každý level obsahuje:

1. TileMapLayer: Pevný terén s kolizemi
2. Player scéna: Instance hlavní postavy
3. Enemies: Skupina instancovaných nepřátel
4. Zóny konce levelu: Area2D uzly, které při detekci hráče spustí přechod vítězného menu.

3.1.3.4 Správa prostředků (Assets)

Všechny herní prostředky jsou v projektu organizovány do složkové struktury.

- /assets: Složka obsahuje veškerou grafiku
- /scenes: Složka obsahuje uložené scény objektů a úrovní.
- /scripts: Složka obsahuje všechny script soubory .gd s kódem.
- /sounds: Složka obsahuje veškeré zvukové efekty použity ve hře.

3.2 Tvorba grafických assetů

Vizuální stránka hry využívá styl pixel art. Grafika pro hru byla tvořena v programu Adobe Photoshop. Každá animace (např. idle, run, jump, shoot) se skládá z několika snímků (frames), které jsou následně v Godotu spojeny do uzlu SpriteFrames.

Obr. 3 Jednotlivé snímky animace pro chůzi a skok

Zdroj: Vlastní

Prostředí hry (země, kameny) jsou tvořeny pomocí Tilesetu. Každá dlaždice má rozměr 16x16 pixelů. TileSet použitý ve hře je vypůjčen a odkaz na něj je ve zdrojích práce.

Animace postavy:

Pro hlavní postavu jsem vytvořil několik sad animací (Sprite Sheets):

1. Idle: Statická poloha
2. Walk: Animace pohybu nohou a rukou
3. Jump: Animace pro skok
4. Attack (Fist): Animace úderu, kde je kladen důraz na přenesení váhy postavy.

3.3 Programování pohybu a animací

Tato kapitola rozebírá skriptování fyziky a logiku vizuální odezvy postavy. Pohyb je implementován v jazyce GDScript v rámci metody **_physics_process(delta).**

Zpracování vstupu z klávesnice a horizontální pohyb

Pro detekci vstupu hráče je využit systém **Input Map** integrovaný přímo v Godotu. Funkce Input.get_axis(„ui_left“, „ui-right“) vrací číselnou hodnotu od -1 (pohyb vlevo) do 1 (pohyb vpravo). Tato hodnota určuje směr, kterým je následně násobena proměnná SPEED. Pokud hráč nestiskne žádnou klávesu, směr zůstane na 0. Pro zajištění plynulého zastavení postavy není využito okamžité vynulování rychlosti, ale metoda move_toward(), která simuluje přirozenou setrvačnost.

Implementace gravitace a skoku

Vertikální pohyb je závislý na gravitaci, která je použitá v každém snímku, pokud postava nestojí na pevné zemi (což ověřuje metoda is_on_floor()). Samotný skok je způsoben přiřazením záporné hodnoty k ose Y (v enginu Godot směřuje osa Y do kladných hodnot směrem dolů.)

```
33 func _physics_process(delta):
34     if is_dead or not input_enabled:
35         velocity = Vector2.ZERO
36         return
37
38     if attack_timer > 0:
39         attack_timer -= delta
40
41     if not is_on_floor():
42         velocity += get_gravity() * delta
43
44     if Input.is_action_just_pressed("ui_accept"):
45         if is_on_floor() and Input.is_action_pressed("ui_down"):
46             position.y += 5
47         elif is_on_floor():
48             velocity.y = JUMP_VELOCITY
49             if jump_sound:
50                 jump_sound.play()
51
52     var direction := Input.get_axis("ui_left", "ui_right")
53     if direction:
54         velocity.x = direction * SPEED
55         sprite.flip_h = direction < 0
56         if attack_area:
57             attack_area.scale.x = -1 if direction < 0 else 1
58     else:
59         velocity.x = move_toward(velocity.x, 0, SPEED)
```

Obr. 4 Ukázka kódu funkce _physics_process(delta)

Zdroj: Vlastní

Propojení s animacemi

Vizuální odezva na pohyb je řešena pomocí uzlu **AnimatedSprite2D.** Skript vyhodnocuje stav fyziky – pokud je postava ve vzduchu, přehraje se animace skoku. Pokud je na zemi a vektor rychlosti není nulový, přehraje se animace chůze. Otáčení postavy do správného směru je vyřešeno úpravou vlastnosti flip_h na hodnotu true nebo false podle toho, zda je směr pohybu menší než nula.

3.4 Detekce kolizí a interakce

Godot k detekci kolizí využívá systém Collision Layers (vrstvy) a Collision Masks (masky).

- Collision Layers: Definují, do jaké skupiny objekt patří. (např. Layer 1: Svět, Layer:2 Hráč)
- Collision Masks: Definují, se kterými vrstvami má daný objekt interagovat

Interakce se světem

Hráči i nepřátelé mají definován **CollisionShape2D,** ve kterém se nachází různé výběry tvarů (např. obdélník, kruh atd.). **CollisionShape2D** je zvolen proto, aby postava hladce klouzala po hranách a nezasekávala se o rohy dlaždic v TileMapě.

Soubojový systém a detekce zásahů:

Střelba je realizována pomocí scény Bullet.tscn. Každý náboj je typu Area2D. Při vstupu jiného objektu do této oblasti (body_entered) dojde k následujícím krokům:

1. Skript zkontroluje, zda objekt patří do skupiny „player“
2. Pokud ano, zavolá se funkce take_damage(), která odečte hodnotu z proměnné zdraví.
3. Projektil je následně odstraněn z paměti pomocí queue_free().

```
7  func _ready():
8      await get_tree().create_timer(lifetime).timeout
9      queue_free()
10
11 func _physics_process(delta):
12     if direction == Vector2.ZERO:
13         return
14     position += direction * speed * delta
15 func _on_body_entered(body):
16     if body.is_in_group("enemy"):
17         return
18
19     if body.is_in_group("player"):
20         if body.has_method("take_damage"):
21             body.take_damage(1)
22         queue_free()
23     else:
24         queue_free()
```

Obr. 5 Ukázka scriptu bullet.gd

Zdroj: Vlastní

3.5 Implementace UI a stavů hry

Uživatelské rozhraní (UI) ukazuje hráči důležité informace, které se hráči hodí vědět. Je postaveno na systému **„Control Nodes“.** Všechny prvky rozhraní jsou umístěny v uzlu CanvasLayer, to je důležité, protože CanvasLayer vytváří samostatnou vrstvu, která není ovlivněna pohybem herní kamery – díky tomu HUD (srdíčka) zůstávají stále na stejném místě obrazovky.

3.5.1 Heads-Up Display (HUD):

Hlavním prvkem je ukazatel zdraví (Health bar). Je využit uzel TextureProgressBar. Tento uzel je v reálném čase propojen se skriptem hráče pomocí signálů. Jakmile hráč utrpí poškození, vyšle se signál health_changed, na který UI reaguje aktualizací hodnoty zdraví.

Obr. 6 Ukázka HUD hry

Zdroj: Vlastní

3.5.2 Systém stavů hry:

Hra má několik základních stavů, které jsou spravovány globálním skriptem:

- **Main Menu**: Úvodní obrazovka se startem hry. V tomto stavu hry se hráči zobrazí počáteční scéna s logem hry v pozadí a třemi tlačítky: Play, Options, Quit. Každé tlačítko má k sobě připojenou metodu pressed, díky které tlačítko ví co má dělat, pokud je zmáčknuto.
- **Active Play**: Standardní průběh úrovně. V tomto stavu se odehrává průběh hry.
- **Death Menu**: Zobrazuje se při nulovém zdraví. V tomto stavu hry se hráči zobrazí text „You Died“ spolu s dvěma tlačítky: RestartButton, BackToMenuButton. Tlačítko RestartButton při stisknutí resetuje běh scény a hra běží znovu od začátku. BackToMenuButton při stisknutí vrátí hráče zpátky do úvodní obrazovky Main Menu.

- **Win Menu:** Zobrazí se po dosažení cílové zóny (Area2D na konci levelu). V tomto stavu hry se hráči zobrazí text „Mission Completed!“ spolu s jedním tlačítkem: NextMissionButton. Toto tlačítko při stisknutí odkazuje na scénu s další úrovní.

Při přechodu mezi těmito stavy je využita metoda **get_tree().change_scene_to_file().** Pro zatraktivnění hry jsem implementoval jednoduché animace, které se zobrazují při určitých stavech hry. Tyto animace jsou vytvořeny pomocí uzlu **AnimationPlayer.**

```
19 func _on_menu_button_pressed():
20     get_tree().paused = false
21     get_tree().change_scene_to_file("res://scenes/main_menu.tscn")
```

Obr. 7 Ukázka kódu pro přepnutí scény (stavu) hry

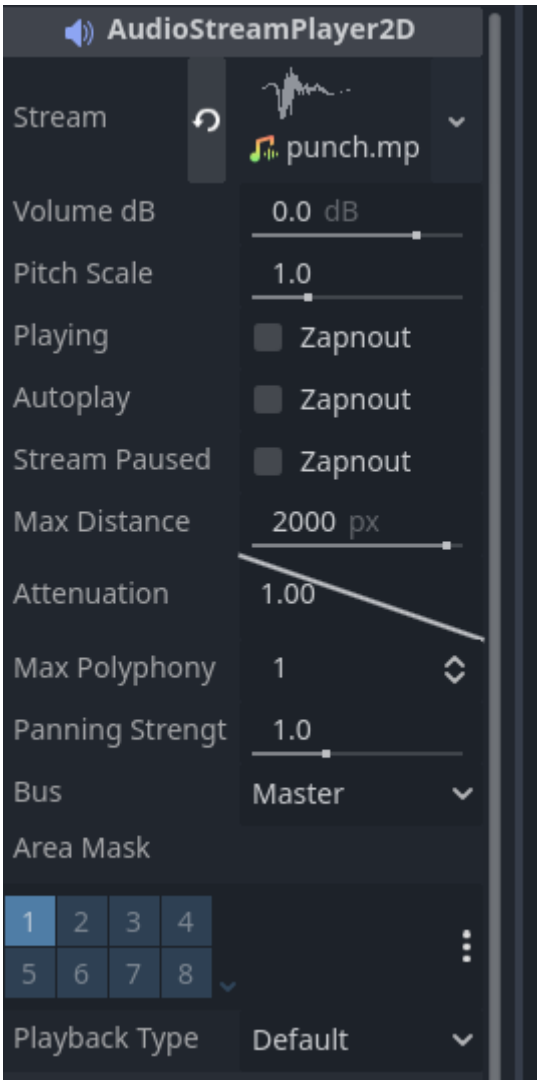
Zdroj: Vlastní

3.6 Implementace audia

Tato kapitola se zabývá správou zvuků. V enginu Godot jsem pro správu zvuků využil uzly AudioStreamPlayer2D.

Technické řešení:

Zvuky jsou do hry vkládány dynamicky pomocí skriptu. Například při skoku je v momentě stisknutí klávesy zavolána metoda jump_sound.play(). Podobně je řešen i zvuk úderu v metodě perform_attack().



Obr. 8 Ukázka uzlu AudioStreamPlayer2D

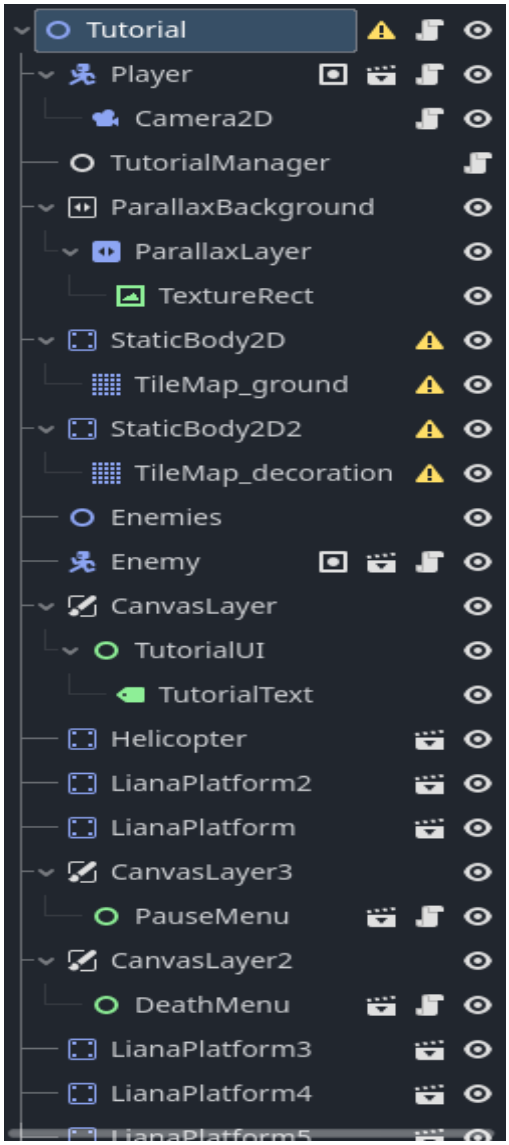
Zdroj: Vlastní

3.7 Systém herních scén a hierarchie uzlů

V enginu Godot jsem využil objektově orientovaný přístup založený na scénách. Každý herní prvek (hráč, nepřítel, páka) je samostatná scéna, kterou lze nezávisle testovat a následně instancovat do hlavní scény úrovně.

Hlavní scéna World.tscn je tvořena stromovou strukturou (Scene Tree). Na nejvyšší úrovni se nachází uzel Node2D, pod kterým jsou seskupeny vrstvy pro pozadí (ParallaxBackground), herní svět (TileMap) a uživatelské rozhraní (CanvasLayer). Tento přístup umožňuje oddělení logiky a vizuální stránky, což usnadňuje pozdější úpravy.

Architektura hry je postavena na struktuře scén. Hlavní scéna (World) slouží jako kontejner, který dynamicky umožňuje upravit např. vzhled uživatelského rozhraní v jedné scéně, aniž by došlo k narušení logiky pohybu hráče v jiné scéně.



Obr. 9 Ukázka spojení uzlu v hlavní scéně

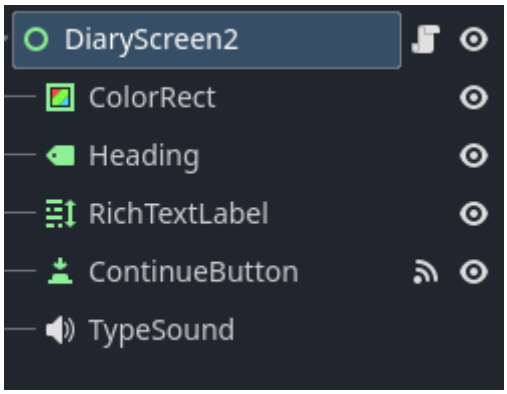
Zdroj: Vlastní

3.8 Implementace systému deníku

Pro prohloubení příběhové linie je vytvořen systém deníkových zápisků. Tento systém využívá samostatnou scénu **DiaryScreen.tscn**, která se aktivuje před začátkem každé úrovně.
Klíčovou funkcí je tzv. typewriter effect (efekt psacího stroje). Namísto toho, aby se text zobrazil okamžitě celý se písmena vykreslují postupně v časovém intervalu. K realizaci je využita kombinace uzlu RichTextLabel a vytvořený časovač (Timer) v GDScriptu

Technické řešení zvuku:

Aby byl zážitek autentičtější, je vykreslení každého znaku doprovázeno krátkým zvukovým efektem. V kódu je použita podmínka, která kontroluje aktuální znak a spouští metodu **.play()** na uzlu AudioStreamPlayer. Díky tomu je zvuk dokonale synchronizován s textem.



Obr. 10 Ukázka scény DiaryScreen.tscn

Zdroj: Vlastní

```
extends Control

@export var next_level_path: String = ""
@export_multiline var diary_text: String = ""

@onready var rich_text_label = $RichTextLabel
@onready var continue_button = $ContinueButton
@onready var type_sound = $TypeSound

var current_char = 0

func _ready():
	rich_text_label.text = diary_text
	rich_text_label.visible_characters = 0
	continue_button.hide()
	continue_button.pressed.connect(_on_continue_pressed)
	var timer = Timer.new()
	add_child(timer)
	timer.wait_time = 0.08
	timer.timeout.connect(_on_timer_timeout)
	timer.start()

func _on_timer_timeout():
	if current_char < diary_text.length():
		current_char += 1
		rich_text_label.visible_characters = current_char

		if diary_text[current_char - 1] != " ":
			type_sound.play()
	else:
		continue_button.show()

func _on_continue_pressed():
	if next_level_path != "":
		get_tree().change_scene_to_file(next_level_path)

```

Obr. 11 Script zajišťující efekt psacího stroje a synchronizaci zvuku

Zdroj: Vlastní

3.9 Implementace dynamických bariér a detekce hráče

V rámci druhé úrovně bylo nutno zajistit průchod hrou a zabránit hráči v návratu zpátky. K tomuto účelu jsem vytvořil systém neviditelných bariér kombinovaných s varováním v podobě textu.

3.9.1 Technické řešení Backtrack Triggeru

Základem je uzel **Area2D** s nastaveným **CollisionShape2D.**

- **Kolizní masky:** Pro správnou funkci bylo nutno nastavit Collision Mask. Trigger je nastaven tak, aby reagoval pouze na vrstvu, ve které se nachází hráč (Layer 2), čímž se zamezilo nechtěné aktivaci s jinými objekty nebo nepřáteli.
- **Funkce Access as Unique Name:** Pro komunikaci mezi triggerem a textovým varováním jsem využil funkci Godotu Access as Unique Name (označeno symbolem %). To umožňuje skriptu najít uzel WarningLabel kdekoli ve stromu scény bez ohledu na jeho přesnou cestu, což zvyšuje stabilitu kódu při změnách v hierarchii levelu.
- **Animace pomocí třídy Tween:** Namísto okamžitého zobrazení textu jsem využil třídu Tween. Ta plynule mění hodnotu průhlednosti (modulate) z 0 na 1 a po krátké prodlevě zpět na 0, což zajišťuje vizuální efekt.

3.10 Funkce export

V této kapitole se věnuji funkci export, která mi dovolila měnit nastavení věcí přímo v editoru, aniž bych musel upravovat věci pořád v kódu.

3.10.1 Nastavení rychlosti střelby

U nepřítele jsem si vytvořil proměnnou shoot_interval a dal před ni @export.

- V tutoriálu jsem nastavil interval střelby na 3 vteřiny, aby měl hráč čas se naučit, co dělat
- V ostatních levelech je interval pozměněn, tak aby to byla pro hráče větší výzva.
  Jedná se o stejný kód, ale nepřítel má více možných intervalů. Díky funkci jsem si ušetřil spoustu času, protože nemusím mít pro každý level jiný skript.

3.11 Konfigurační nastavení systému

V rámci vývoje aplikace bylo využito několik konfiguračních nastavení, která ovlivňují chování hry.

- **Input Map**

Ovládání hráče bylo definováno pomocí systému Input Map v enginu Godot. Byly vytvořeny akce jako:

- pohyb vlevo a vpravo
- skok
- útok
- interakce

- **Collision Layers a Masks**

Pro správnou detekci kolizí byly nastaveny kolizní vrstvy:

- hráč
- nepřátelé
- prostředí

Tím bylo zajištěno, že objekty reagují pouze na relevantní prvky.

- **Export nastavení**

Při exportu hry byly nastaveny parametry pro cílovou platformu Windows, včetně:

- rozlišení okna
- režimu zobrazení
- optimalizace výkonu

- **Exportované proměnné**

Pomocí anotace @export byly definovány proměnné, které lze upravovat přímo v editoru, například:

- rychlost střelby nepřátel
- pohybové parametry

4 UŽIVATELSKÁ ČÁST

4.1 Návod ke spuštění hry

Hra Lone Survivor je vytvořená pro operační systém Windows. Hra nevyžaduje složitou instalaci. Pro spuštění hotového buildu je potřeba dodržet následující kroky:

1. Otevřete repozitář projektu na platformě GitHub
2. Přejděte do sekce Releases
3. Vyberte nejnovější verzi hry
4. Stáhněte soubory Lone Survivor.exe a Lone Survivor.pck
5. Oba soubory uložte do stejné složky (např. *LoneSurvivor*).
6. Spusťte soubor **Lone Survivor.exe**

Možné problémy při spuštění:

- Pokud systém zobrazí bezpečnostní upozornění, je nutné zvolit možnost „Přesto spustit“.
- V případě, že se hra nespustí, je nutné zkontrolovat, zda se soubor .pck nachází ve stejné složce jako .exe.

Aktuální verze hry je dostupná v sekci Releases.

4.2 Uživatelské rozhrání po spuštění hry

Po spuštění hry se hráči zobrazí hlavní menu, na kterém hráč může stisknout tlačítko pro spuštění tutoriálu. Po kliknutí na tlačítko se hráči zobrazí scéna s zápiskem deníku hlavní postavy, kde je vysvětlen základní příběh. Po stisknutí tlačítka CONTINUE hráč pokračuje do samotného tutoriálu. Pro úspěšné přežití je nutné sledovat vizuální prvky na obrazovce:

- Ukazatel zdraví (vlevo nahoře): Pět červených srdcí znázorňuje zbývající životy vojáka. Jakmile dojde ke kontaktu s nábojem vystřeleným od nepřítele, srdce zmizí a hráčovi ubyde život. Pokud zmizí všechna srdíčka a hráč tak bude mít nula životů, hra končí.
- Interaktivní prvky: V prostředí se nachází páky a lékárničky. Pokud hráč dojde k páčce, může provést akci. Po dotknutí lékárničky se hráči automaticky doplní životy na plnou hodnotu.
- Pokud hráč zemře, zobrazí se varování „YOU DIED“ s možností restartovat úroveň

4.2.1 Main menu hry

Hráč se po spuštění hry ocitá v hlavním menu hry. Main menu obsahuje tři tlačítka, díky kterým hráč může pokračovat hrou:

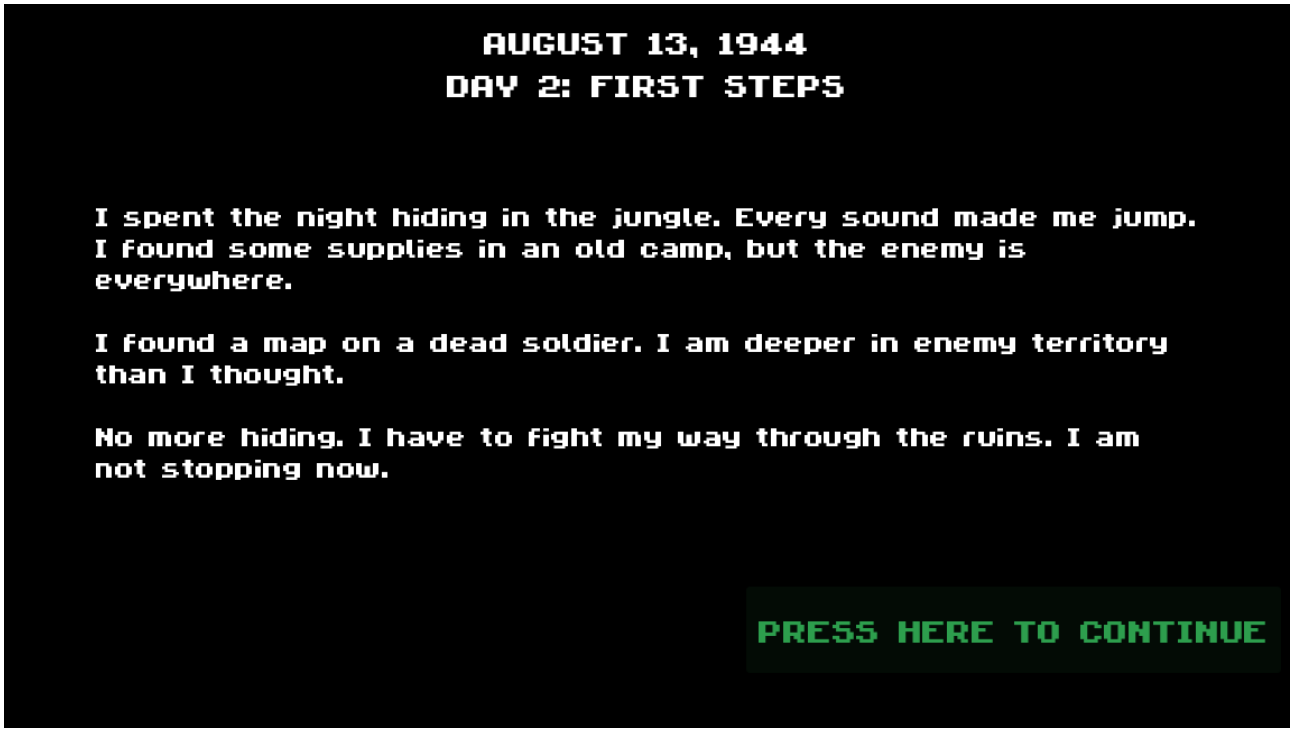
- **Play button:** Hráč po stisknutí tlačítka play se ocitá v tutoriálu hry
- **Options button:** Hráči se po stisknutí tlačítka zobrazí základní ovládání hry
- **Quit button:** Hráč po stisknutí tlačítka opustí hru

Obr. 12 Ukázka Main Menu hry

Zdroj: Vlastní

4.2.2 Diary menu

Hráči se po stisknutí tlačítka play zobrazí černá obrazovka s deníkem (viz. Obr. 16). Text informuje hráče o tom, že je jediným přeživším po pádu vrtulníku a jeho cílem je najít základnu. Hráč nemůže postoupit dál, dokud se text nedopíše, což zajišťuje, že mu neunikne žádná část příběhu. Po dopsaní se objeví tlačítko CONTINUE.



Obr. 13 Ukázka deníku

Zdroj: Vlastní

4.2.3 Controls menu

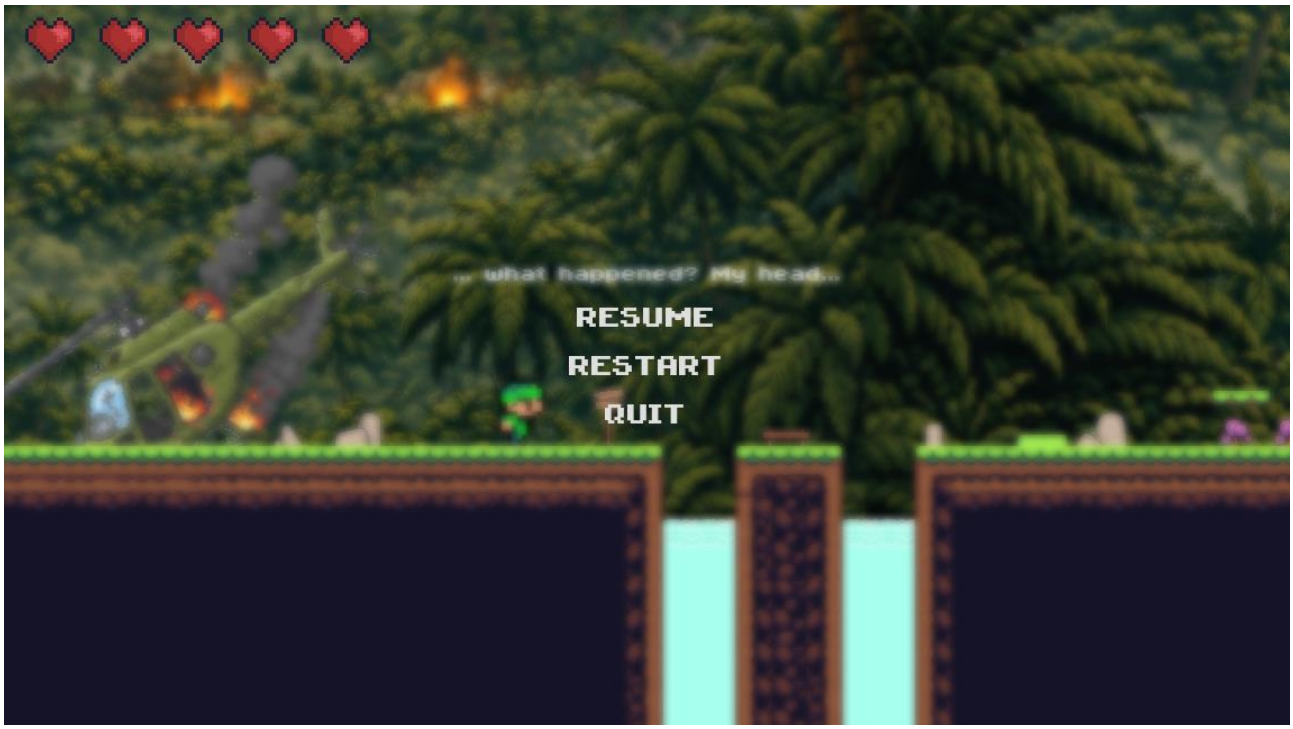
Po stisknutí tlačítka options se hráč přesune do sekce ovládání. Obrazovka obsahuje všechny základní mechaniky, které bude hráč v průchodu hrou potřebovat. Nachází se zde všechny klávesy, které hráč použije (pohyb pomocí šipek, skok mezerníkem, útok klávesou K a interakce s pákou klávesou E). Zpět do hlavního menu se hráč může dostat stisknutím tlačítka BACK TO MENU.

Obr. 14 Ukázka obrazovky CONTROLS

Zdroj: Vlastní

4.2.4 Pause menu

Během hraní hry může hráč kdykoliv stisknout klávesu ESC, čímž vyvolá Pause menu hry. Čas ve hře se kompletně zastaví, což dá prostor hráči pro AFK (Away from keyboard). Z tohoto menu lze buď pokračovat ve hře, nebo se vrátit zpět na úvodní menu nebo zavřít hru.



Obr. 15 Ukázka Pause Menu

Zdroj: Vlastní

4.2.5 Win/Die menu

Hra reaguje na hráčův pokrok hrou dvěma způsoby, buď při smrti se zobrazí Death Menu, nebo při dokončení úrovně se zobrazí Win Menu.

- **Death Menu:** Pokud hráč ztratí všechny životy, nebo spadne do vody, obrazovka zčervená a objeví se nápis „YOU DIED“. Hráč má možnost úroveň ihned restartovat pomocí tlačítka RESTART, nebo se vrátit do úvodního menu.
- **Win Menu:** Jakmile se hráč úspěšně dostane na konec úrovně a dotkne se červené vlajky, úroveň se uzamkne a vyskočí text „Mission Completed!“ Tlačítko NEXT MISSION následně hráče plynule přesune do další úrovně.

Obr. 16 Ukázka Death Menu hry

Zdroj: Vlastní

Obr. 17 Ukázka Win Menu hry

Zdroj: Vlastní

4.3 Průchod hrou

V této kapitole popíšu obecně průchod tutoriálem.

1. **Startovní zóna:** Hráč začíná u trosek vrtulníku. Zde se seznámí s pohybem (šipka doleva, šipka doprava) a skokem (mezerník).

Obr. 18 Startovní zóna

Zdroj: Vlastní

2. **První překážka:** První překážka je vodní plocha před hráčem. Zde hráč zjišťuje, že pád do hluboké vody je automatickou smrtí.

Obr. 19 První překážka (vodní plocha)

Zdroj: Vlastní

3. **První střet s nepřítelem:** Hráč uvidí hlídkujícího vojáka na plošině. Hráč musí využít útok (klávesa K), aby nepřítele zneškodnil dříve, než ho nepřítel zabije.

Obr. 20 První střet s nepřítelem

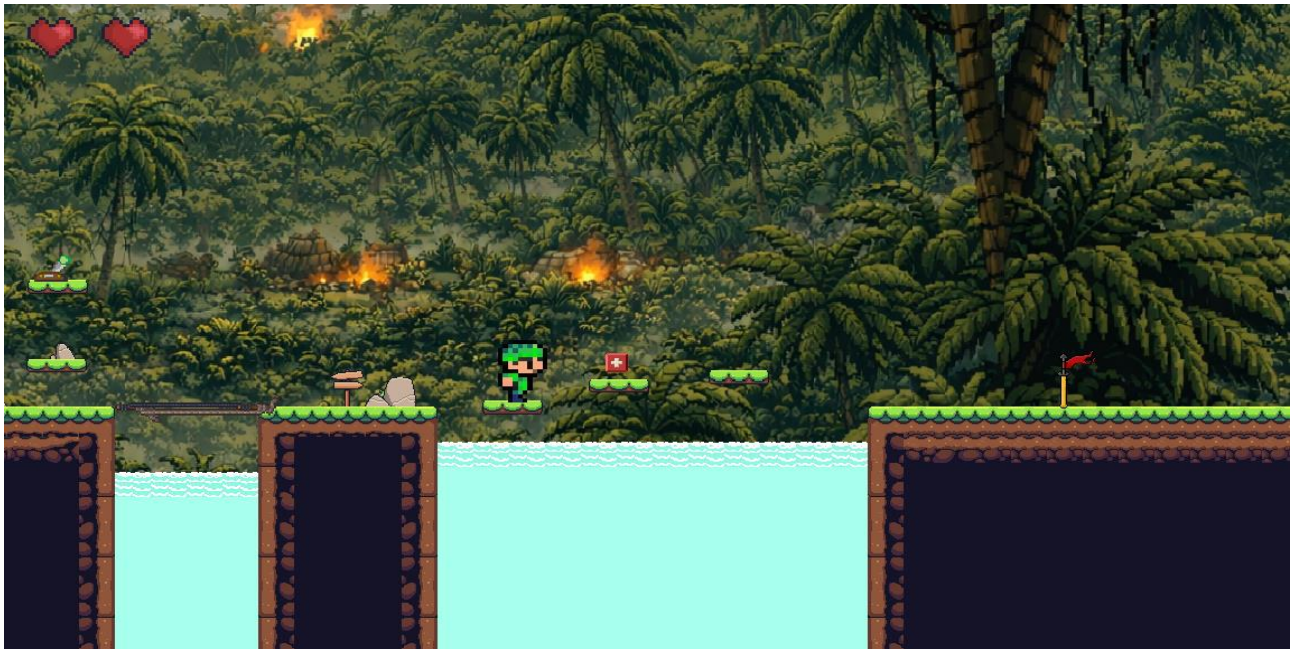
Zdroj: Vlastní

4. **Seznámení s padacími mosty a páčkami:** Hráč po zabití nepřítele se dostane k menšímu parkouru. Na konci tohoto parkouru se nachází páčka, které se pokud hráč dotkne, zjistí, že pomocí klávesy E jí může stisknout, po stisknutí páčky se spustí padací most a hráč může pokračovat dále.

Obr. 21 Seznámení s padacím mostem a páčkou

Zdroj: Vlastní

5. **Seznámení s doplněním životů pomocí lékárničky:** Hráč po spuštění páčky a spadnutí mostu pokročí dále a uvidí na platformě lékárničku, po sebrání lékárničky se hráči doplní životy na maximální úroveň.



Obr. 22 Ukázka lékárničky

Zdroj: Vlastní

6. **Seznámení s koncem úrovně:** Hráč po dokončení parkouru přes platformy dojde k vlajce, které když se dotkne, úroveň se splní a hráč může pokračovat na další úroveň, teď už bez tutoriálu.

Obr. 23 Ukázka konce úrovně

Zdroj: Vlastní

4.4 Minimální systémové požadavky

Pro plynulý chod hry Lone Survivor jsou doporučeny následující hardwarové a softwarové parametry:

- **Operační systém:** Windows 10 / 11 (64-bit)
- **Procesor:** Dvoujádrový procesor s frekvencí 2.0 GHz a vyšší
- **Operační paměť:** 4 GB RAM
- **Grafická karta:** Integrovaná grafická karta s podporou OpenGL 3.3
- **Místo na disku:** cca. 100 MB volného místa

5 TESTOVÁNÍ

Cílem této kapitoly bylo odhalit chyby hry tzv. bugy, ověřit stabilitu hry a zjistit, zda je obtížnost vyvážená pro různé typy hráčů.

5.1 Metodika testování

Testování probíhalo tak, že testeři neměli přístup ke zdrojovému kódu, ale hru pouze hráli a snažili se v ní najít nedostatky. Zaměřil jsem se na testování herních mechanik (skok, útok, interakce), funkčnost uživatelského rozhraní (UI) a plynulost přechodů mezi úrovněmi.

Testovací scénáře

Každý tester měl definovaný úkol:

- **Tester 1:** projít hru bez pomoci a hodnotit obtížnost
- **Tester 2:** ověřit srozumitelnost ovládání
- **Tester 3:** pokusit se najít chyby (tzv. destruktivní testování)

Testování probíhalo formou praktického hraní bez přístupu ke zdrojovému kódu.

5.1.1 Tester 1 (Zkušený hráč, spolužák)

První tester měl bohaté zkušenosti s 2D plošinovkami. Tutoriál prošel bez problémů, ale upozornil na to, že v pozdějších fázích hry je obtížnost příliš nízká, protože nepřátelé střílí příliš pomalu. Také se pokoušel vrátit za začátek mapy.

5.1.2 Tester 2 (Začátečník, příbuzný)

Druhý tester nehraje hry pravidelně. Jeho testování bylo klíčové pro ověření, zda dokáže porozumět ovládání. Tester měl ze začátku problém s načasováním skoků a stěžoval si, že ho nepřátelé zabili příliš rychle již v tutoriálu.

5.1.3 Tester 3 (Vývojář her)

Třetí tester dostal úkol zaměřit se na „rozbíjení“ hry. Snažil se klikat na tlačítka v menu rychle, mačkat více kláves najednou a přeskakovat text v deníku. Zjistil, že pokud se text v deníku nedopíše do konce, může dojít k přeskočení příběhu.

5.2 Odstranění chyb (bugů)

Na základě zpětné vazby od testerů jsem do hry implementoval několik důležitých oprav a vylepšení:

1. **Zabránění backtrackingu (návratu):** Jelikož první tester odhalil, že se hráč může vracet zpět, za hranici mapy a tím narušit celý level, rozhodl jsem se použít za začátek levelu neviditelnou kolizi, která brání hráči v průchodu. Problém jsem vyřešil implementací uzlu Area2D s kolizní maskou, která detekuje hráče a zobrazí mu varování, že se nemůže vrátit (popsáno v podkapitole 3.9)
2. **Vyvážení obtížnosti:** Abych vyhověl jak začátečníkům, tak zkušeným hráčům, přidal jsem do kódu exportovanou proměnnou shoot_interval (podkapitola 3.10). Díku tomu má nepřítel v tutoriálu dlouhou prodlevu mezi střelami (3 sekundy), zatímco v ostatních úrovních je agresivnější.
3. **Oprava přeskakování deníku:** Zamezil jsem hráči v přeskočení textu tím, že tlačítko „CONTINUE“ se v deníku zobrazí až ve chvíli, kdy časovač (Timer) kompletně vypíše celý příběhový text.

Identifikovaný problém	Příčina chyby	Zvolené řešení
Hráč se mohl vracet na začátek mapy.	Chybějící bariéra v level designu.	Implementace Area2D triggeru s varovným textem.
Příliš vysoká obtížnost pro začátečníky.	Pevně nastavený interval střelby AI.	Parametrizace pomocí @export var proměnné.
Předčasné ukončení deníku.	Tlačítko CONTINUE bylo aktivní ihned.	Skrytí tlačítka do doby, než Timer dopíše text.

Tab. 1 Přehled opravených chyb během testování

5.3 Výsledek testování

Po opravení všech chyb prošla hra finálním testováním, při kterém nebyly objeveny žádné kritické chyby, které by způsobovaly nehratelnost hry. Hra běží plynule (60 FPS) a všechny herní smyčky (od zapnutí menu až po vítěznou obrazovku) na sebe správně navazují.

6 REPOZITÁŘ S ŘEŠENÍM PRÁCE

Kompletní zdrojové kódy, herní assety i exportovaný build hry (.exe soubor) jsou nahrány a veřejně dostupné na platformě GitHub.

Odkaz na repozitář: https://github.com/StanislavVacula/Maturitni_projekt_hra_Lone_Survivor

ZÁVĚR

Výsledkem této maturitní práce je plně funkční a spustitelná 2D akční videohra s názvem „Lone Survivor“, vytvořená v moderním herním engine Godot 4.4. Jedná se o titul žánru „platformer“, který kombinuje mechaniky pohybu v terénu s akčním soubojovým systémem. Hra obsahuje kompletní herní smyčku, animace postav vytvořené v softwaru Pixelorama a Photoshop a inteligentní protivníky. Po technické stránce produkt využívá objektově orientovanou architekturu, implementuje dynamickou detekci kolizí a umělou inteligenci nepřátel, která reaguje na přítomnost hráče v reálném čase.

Všechny cíle projektu se podařilo splnit. Úspěšně se podařilo vytvořit aplikaci běžící na platformě Windows, která obsahuje propracovaný systém ovládání a mechaniky boje. V zadání bylo specifikováno vytvoření základního pohybu a interakce, což bylo v konečné fázi dosaženo implementací umělé inteligence nepřátel. Ti využívají k detekci hráče RayCast2D, který simuluje zorné pole a díky tomu zabraňuje nepřátelům hráče vidět skrz pevné objekty.

Během práce na projektu jsem učinil několik důležitých technických rozhodnutí. Jedno z klíčových rozhodnutí, byla volba správné třídy pro fyziku postav, k čemuž jsem využil metodu CharacterBody2D, která s vestavěnou metodou move_and_slide() zajistila stabilitu kolizí a plynulý pohyb po šikmých plochách. Toto rozhodnutí odstranilo problémy s propadáváním texturami a zasekávání mezi objekty.

Produkt splnil všechna původní očekávání. Hra obsahuje jak všechny plánované prvky, jako je boj a interakce s TileMapou, ale podařilo se mi i vytvořit funkční a plynulé animace, což nebylo v zadání plánováno.

Práce na projektu mi přinesla hluboké znalosti v oblasti tvorby her a skriptování. Projekt nabízí široké spektrum možností pro budoucí rozšíření, jako je vytvoření více úrovní, více zbraní, vylepšená AI nepřátel atd.

SEZNAM POUŽITÉ LITERATURY

(1) PLAYWITHFURCIFER. *How to Use the New TileMap in Godot 4*. Online. 2022. Dostupné z: https://www.youtube.com/watch?v=tQSL2scuqeU. [cit. 2025-11-25].

(2) LINIETSKY, Juan a MANZUR, Ariel. *Godot Docs*. Online. © Copyright 2014-present. Dostupné z: https://docs.godotengine.org/en/stable/. [cit. 2025-11-25].

(3) LINIETSKY, Juan; MANZUR, Ariel a GODOT COMMUNITY. *Documentation 2D tutorials*. Online. 2014. Dostupné z: https://docs.godotengine.org/en/stable/tutorials/2d/index.html. [cit. 2025-12-09].

(4) SAULTOONS. *The Ultimate Pixel Art Tutorial*. Online. 2021. Dostupné z: https://www.youtube.com/watch?v=lfR7Qj04-UA. [cit. 2025-12-09].

(5) FIRESHIP. *Godot in 100 Seconds*. Online. 2022. Dostupné z: https://www.youtube.com/watch?v=QKgTZWbwD1U&t. [cit. 2025-12-09].

(6) CODE, Coco. *Start Your Game Creation Journey Today! (Godot beginner tutorial)*. Online. 2024. Dostupné z: https://www.youtube.com/watch?v=5V9f3MT86M8. [cit. 2026-01-20].

(7) BRACKEYS. *How to program in Godot - GDScript Tutorial*. Online. 2024. Dostupné z: https://www.youtube.com/watch?v=e1zJS31tr88. [cit. 2026-01-20].

(8) SWYDEV. *MASTER 2D Movement in Godot - Everything You Need To Know*. Online. 2024. Dostupné z: https://www.youtube.com/watch?v=RDT5hfYWCrQ. [cit. 2026-01-20].

(9) DEVWORM. *Full Guide To 2D Collisions in Godot 4*. Online. 2023. Dostupné z: https://www.youtube.com/watch?v=GiZuWjXmvcc. [cit. 2026-01-22].

(10) GAMESTICK. *Simple Shooting system in Godot4 2D | godot tutorial*. Online. 2024. Dostupné z: https://www.youtube.com/watch?v=d8a5xaxSWks. [cit. 2026-01-23].

(11) GDQUEST. *All There Is To Know About Godot's Scenes and Nodes*. Online. 2021. Dostupné z: https://www.youtube.com/watch?v=y0QAvyv9Wbw. [cit. 2026-01-23].

(12) GDQUEST. *Getting Started With Godot's Scene Instances*. Online. 2021. Dostupné z: https://www.youtube.com/watch?v=_ImAgihyy3A. [cit. 2026-01-23].

(13) PAPER MOUSE GAMES. *2D Platformer Tutorial In Godot 4*. Online. 2024. Dostupné z: https://www.youtube.com/playlist?list=PLCcur7_Y2zTdKIQ2oM2Ec8MEfeBnAbEXT. [cit. 2026-02-19].

(14) GDQUEST. *Make Your First 2D Game with Godot: Player and Enemy (beginner tutorial part 1)*. Online. 2019. Dostupné z: https://www.youtube.com/watch?v=Mc13Z2gboEk. [cit. 2026-02-19].

(15) BLACKSPIRE. *Nature pixel art base assets FREE*. Online. 2019. Dostupné z: https://assetstore.unity.com/packages/2d/environments/nature-pixel-art-base-assets-free-151370. [cit. 2026-02-22].

(16) DASHNOTHING. *Godot @export - Everything You Need to Know*. Online. 2024. Dostupné z: https://www.youtube.com/watch?v=VOcN6Y8mTEE. [cit. 2026-02-23].

(17) THE SHAGGY DEV. *Scene transition effects in Godot 3*. Online. 2022. Dostupné z: https://www.youtube.com/watch?v=yZQStB6nHuI. [cit. 2026-02-23].

(18) CODE IT ALL. *Godot Shaders : An Introduction*. Online. 2025. Dostupné z: https://www.youtube.com/watch?v=JM09avtMlmE. [cit. 2026-02-24].

(19) GWIZZ. *Godot 4 Tween Tutorial*. Online. 2023. Dostupné z: https://www.youtube.com/watch?v=GN_HMhvYnYA. [cit. 2026-02-24].

(20) ADOBE ASIA PACIFIC. *How to Make Pixel Art [Tutorial for Beginners] | Adobe Photoshop*. Online. 2021. Dostupné z: https://www.youtube.com/watch?v=9OfeNDukXm8. [cit. 2026-02-26].

(21) OPENAI. *Ahoj, pracuji na dokumentaci závěrečného maturitního projektu, chci aby si si soubor celý prošel, zkontroloval hrubky, zkontroloval názvy podkapitol a kapitol, zda to sedí, dále zkontrolovat zda je dokumentace v pořádku, a kdyby náhodou doplnit o co dokumentaci doplnit/co vylepšit*. IN: ChatGPT-5 Online. 22. března 2026. Dostupné z: https://chatgpt.com/share/69c1857b-cb04-800d-9ae9-493041e3d28f. [cit. 2026-03-21].

SEZNAM OBRÁZKŮ

Obr. 1 Ukázka stromu scén u hlavní postavy 14
Obr. 2 Použitý TileSet 15
Obr. 3 Jednotlivé snímky animace pro chůzi a skok 16
Obr. 4 Ukázka kódu funkce _physics_process(delta) 18
Obr. 5 Ukázka scriptu bullet.gd 19
Obr. 6 Ukázka HUD hry 20
Obr. 7 Ukázka kódu pro přepnutí scény (stavu) hry 21
Obr. 8 Ukázka uzlu AudioStreamPlayer2D 22
Obr. 9 Ukázka spojení uzlu v hlavní scéně 23
Obr. 10 Ukázka scény DiaryScreen.tscn 24
Obr. 11 Script zajišťující efekt psacího stroje a synchronizaci zvuku 25
Obr. 12 Ukázka Main Menu hry 29
Obr. 13 Ukázka deníku 30
Obr. 14 Ukázka obrazovky CONTROLS 31
Obr. 15 Ukázka Pause Menu 32
Obr. 16 Ukázka Death Menu hry 33
Obr. 17 Ukázka Win Menu hry 34
Obr. 18 Startovní zóna 34
Obr. 19 První překážka (vodní plocha) 35
Obr. 20 První střet s nepřítelem 35
Obr. 21 Seznámení s padacím mostem a páčkou 36
Obr. 22 Ukázka lékárničky 37
Obr. 23 Ukázka konce úrovně 37

SEZNAM TABULEK

Tab. 1 Přehled opravených chyb během testování 40